



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Laboratorio di Sicurezza Informatica

Linux Packet Filter

Marco Prandini

Dipartimento di Informatica – Scienza e Ingegneria

IPTABLES

Il packet filter di default nei sistemi Linux fino al 2014

- può ancora essere necessario saperlo gestire
 - sistemi legacy
 - usato indirettamente da altri strumenti
- è più semplice e utile per introdurre i concetti base

Documentazione

- man iptables (sintassi del comando)
<https://ipset.netfilter.org/iptables.man.html>
- man iptables-extensions (opzioni dei criteri di match e azioni)
<https://ipset.netfilter.org/iptables-extensions.man.html>

gestione delle catene

sintassi base del comando

`iptables [-t <tabella>] -CMD [catena] [match] [-j <target>]`

se omesso si assume -t filter

comandi (*CMD*) principali:

- L **list** elenca le regole della catena (se presente, o tutte)
- C **check** ritorna true se una regola coi match e il target specificati esiste nella catena
- A **append** aggiunge una regola in fondo alla catena
- I [n] **insert** inserisce una regola (in n-esima posizione, o in testa se manca n)
- D [n] **delete** rimuove una regola (in n-esima posizione, o quella che ha esattamente i match e il target specificati)
- R n **replace** sostituisce la regola in n-esima posizione
- F **flush** svuota la catena
- P **policy** imposta la policy di default della catena

match di base sull'header IPv4

Caratteristiche “Layer 1”

- i <input interface>
- o <output interface>

Caratteristiche “Layer 2”

- solo caricando l'estensione con `-m mac`
 - `--mac-source <source mac address>`

Caratteristiche “Layer 3”

- s <source address>
- d <destination address>
- f (fa match coi frammenti dal secondo in poi)
- solo caricando l'estensione con `-m iprange`
 - `--src-range from-to`
 - `--dst-range from-to`

innumerevoli altre:
`man iptables-extensions`

match di base sull'header IPv4

Caratteristiche “Layer 4”

`-p <tcp|udp|udplite|icmp|icmpv6|esp|ah|sctp|mh>`

se il protocollo supporta le porte, abilita l'interpretazione di

`--dport port[:port]`

`--sport port[:port]`

se il protocollo è tcp, abilita l'interpretazione di

`--tcp-flags mask comp`

- **i flag** sono SYN ACK FIN RST URG PSH *ALL NONE*
- **mask** = elenco flag “interessanti” (gli altri flag del pacchetto sono ignorati)
- **comp** = elenco flag tra quelli interessanti che devono essere settati per fare match

se il protocollo è icmp, abilita l'interpretazione di

`--icmp-type <type>`

- **elenco tipi:** `iptables -p icmp -h`

esempi

```
iptables -A FORWARD -i ppp0 -d 87.15.12.0/24 -p tcp --dport 80  
-j ACCEPT
```

```
iptables -P INPUT DROP
```

```
iptables -t nat -A POSTROUTING -s 192.168.0.0/24 -o ppp0  
-j SNAT --to-source 87.4.8.21
```

```
iptables -t nat -A PREROUTING -i ppp0 -d 87.4.8.21 -p tcp  
--dport 2222 -j DNAT --to-destination 192.168.0.1:22
```

```
iptables -D FORWARD 1
```

```
iptables -I INPUT 13 -p icmp ! --icmp-type echo-reply -j DROP
```

```
iptables -A OUTPUT -p tcp --tcp-flags SYN,ACK,FIN FIN
```

terminating target specifici della tabella nat

nelle catene PREROUTING o OUTPUT

- il target **DNAT** indica a netfilter che deve essere modificato l'indirizzo di destinazione del pacchetto
- l'opzione `--to-destination [ipaddr[-ipaddr]][:port[-port]]` permette di specificare la nuova destinazione
- il target **REDIRECT** funziona come **DNAT** ma è necessario se si vuole specificamente ridirigere il pacchetto alla macchina locale
- l'opzione `--to-ports` permette di cambiare la porta di destinazione

nelle catene POSTROUTING o INPUT

- il target **SNAT** indica a netfilter che deve essere modificato l'indirizzo di sorgente del pacchetto
- l'opzione `--to-source [ipaddr[-ipaddr]][:port[-port]]` permette di specificare la nuova sorgente
- il target **MASQUERADE** (valido solo in **POSTROUTING**) funziona come **SNAT** assegnando automaticamente al pacchetto l'indirizzo dell'interfaccia di uscita

se vengono utilizzati intervalli, di default, i diversi indirizzi e porte vengono utilizzati a turno (round-robin) via via che arrivano pacchetti

gestione dei contatori

I contatori vengono visualizzati col comando `list (-L)`

- se unito all'opzione `-v`
- in forma “human readable” (K, M, G)
a meno che non si usi l'opzione `-x`

I contatori possono essere azzerati col comando `-Z`

Per una lettura reiterata precisa, è importante svolgere lettura e azzeramento in modo contestuale

```
iptables -vnxL -Z > contatori
```

invece di

```
iptables -vnxL > contatori
```

```
iptables -Z
```

- nel tempo che trascorre tra i due comandi,
possono arrivare pacchetti
- non inclusi nella lettura del primo comando
 - azzerati prima che il ciclo di lettura si ripeta

catene custom

registrare nuovi hook è complesso, ma iptables offre un'alternativa semplice: creare catene custom

- `iptables [-t TAB] -N <NOME>`
 - crea la catena **NOME** nella tabella **TAB**
 - la catena è ignorata fintanto che non ci si “salta dentro” da una catena builtin:
- `iptables [-t TAB] -A <BC> ...match... -j <NOME>`
 - regola inserita nella catena builtin **BC** della tabella **TAB**
 - se c'è match, il pacchetto salta alla catena **NOME**
 - **NOME** e **BC** devono essere definite nella stessa tabella
 - iptables inizia a scorrere le regole di **NOME**, che sono gestibili in modo identico a quello delle catene builtin, con un'eccezione:
- `iptables [-t TAB] -A <NOME> -j RETURN`
 - termina lo scorrimento delle regole di **NOME** e ritorna alla catena **BC** da cui era stato fatto il salto, riprendendo l'esame delle regole da quella successiva
- `iptables [-t TAB] -X <NOME>`
 - rimuove la catena **NOME** dalla tabella **TAB**
 - funziona solo se non ci sono salti verso **NOME** in altre catene

utilità delle catene custom – esempi

tenere in ordine set di regole molto ampi creando gerarchie di classificazione

- per finalità: filtraggio vs. monitoraggio
- per località: sottoreti, protocolli, ...
- per tempo di vita: regole persistenti vs. temporanee
- set di regole predefinite “plug-in” scelte da librerie vs. personalizzazioni

approccio usato da vari framework per la gestione di firewall, es. ufw, shorewall, ...

utilità delle catene custom – esempi

rendere più semplice e robusta la gestione di set di regole dinamici

- es. creando catene custom quando appare in rete una nuova entità da gestire
 - un solo salto dalle catene built-in → pulizia organizzativa
 - facilmente individuabili → semplicità di monitoraggio

esempio: per ogni server web rilevato dinamicamente a valle del router-firewall

```
iptables -N SRV_$IPSERVER
```

```
iptables -I FORWARD -d $IPSERVER -j SRV_$IPSERVER
```

```
iptables -I FORWARD -s $IPSERVER -j SRV_$IPSERVER
```

inserimento in SRV_\$IPSERVER di regole di filtraggio di base e regole dinamicamente aggiunte e tolte per affrontare situazioni come DoS, account bruteforcing, ecc.

- allo spegnimento del server, indipendentemente da quante regole sono via via state aggiunte alla catena custom, basterà:

```
iptables -F SRV_$IPSERVER
```

```
iptables -D FORWARD -d $IPSERVER -j SRV_$IPSERVER
```

```
iptables -D FORWARD -s $IPSERVER -j SRV_$IPSERVER
```

```
iptables -X SRV_$IPSERVER
```

NFTABLES

È il sistema correntemente implementato nel kernel, con un'interfaccia utente che rimpiazza non solo iptables, ma tutti comandi della stessa suite

- **arptables** per gestire il filtraggio delle richieste di risoluzione IP→ MAC
- **ip6tables** per filtrare il traffico Ipv6
- **ebtables** per costruire bridge (data link layer) con funzionalità di filtraggio

Vantaggi:

- gestione integrata e omogenea del traffico a tutti i livelli dello stack
- prestazioni elevate grazie al superamento del modello di analisi lineare delle catene, sostituito da mappe e concatenazioni
- spostamento della complessità di gestione dal kernel agli strumenti userspace → semplificazione degli aggiornamenti

Credits

- molti degli esempi seguenti sono tratti dal [tutorial originale di P. N. Ayuso](#)
- Riferimento essenziale [Quick reference-nftables in 10 minutes](#) e tutto il [Wiki](#)

nft - tabelle

Non esistono tabelle predefinite con significato assegnato.
Comandi per gestirle:

```
nft list tables [<family>]
```

elenca le tabelle
definite per *family*

```
nft [-n] [-a] list table [<family>] <name>
```

elenca le regole
nella tabella *name*

```
nft (add | delete | flush) table [<family>] <name>
```

in cui family è uno tra **ip** (il default), **arp**, **ip6**, **bridge**,
inet, **netdev**

La prima azione per configurare un firewall sarà quindi ad es.

```
nft add table ip foo
```

nft - catene

Le catene sono i contenitori fondamentali delle regole, e valgono i principi generali introdotti per l'architettura di netfilter e illustrati con gli esempi iptables.

Non esistono catene predefinite. Alla loro creazione:

- vanno collocate in una tabella **table**
- va stabilito un tipo di manipolazione (com'era per i nomi delle tabelle iptables) → **proprietà type della catena**
- va stabilito il momento in cui le regole devono essere esaminate (com'era definito dai nomi delle catene iptables) → **hook a cui si aggancia una catena base**
- si deve specificare una **priority** per ordinarne l'invocazione
 - tra catene custom agganciate allo stesso hook
 - rispetto a operazioni strutturali di netfilter
- si può direttamente specificare la **policy** di default

type:

filter
route
nat

hook:

prerouting
input
forward
output
postrouting

nft - catene

il backslash è usato per dire che `{}`; sono da inserire letteralmente, non sono marcatori sintattici del manuale (es. opzioni alternative)

Sintassi generale:

```
nft (add | create) chain [<family>] <table> <name>
[ \{ type <type> hook <hook> [device <device>] priority
<priority> \; [policy <policy> \;] \} ]
```

```
nft (delete | list | flush) chain [<family>] <table> <name>
```

```
nft rename chain [<family>] <table> <name> <newname>
```

Esempio

```
nft 'add chain ip foo bar {
    type filter hook input priority 0; policy drop;
}'
```

a loro volta, i caratteri speciali e gli “a capo” vanno protetti da per farli interpretare letteralmente e non col significato speciale che avrebbero per la shell; l'intero comando può essere passato a nft come un unico parametro

nft - regole

Le regole vengono inserite nelle catene e specificano come noto

- criteri di match
- statement da eseguire

Sintassi:

```
nft add rule [<family>] <table> <chain> <matches> <statements>
nft insert rule [<family>] <table> <chain> [position <handle>]
<matches> <statements>
nft replace rule [<family>] <table> <chain> [handle <handle>]
<matches> <statements>
nft delete rule [<family>] <table> <chain> [handle <handle>]
```

Esempio:

```
nft add rule ip foo bar ct state established,related accept
nft add rule ip foo bar ct state new tcp dport 22 accept
```


nft – match comuni per layer data/network

```
ether saddr /daddr <mac_address>
```

```
ip protocol { icmp, esp, ah, comp, udp,  
udplite, tcp, dccp, sctp }
```

```
ip saddr / daddr <addr_spec>
```

– *addr_spec* può essere

- IP
- subnet/netmask
- calcolo con bitmask e confronto, es. `& 0.0.0.255 < 0.0.0.127`
- range IPstart-IPend
- elenco { IP1, IP2, ... }
- negato con `!= addr_spec`

nft – match comuni per tcp/udp/icmp

```
tcp sport / dport <port_spec>
```

- port_spec può essere
 - PORTA
 - range Pstart-Pend
 - elenco { P1, P2, ... }
 - negato con `!= port_spec`
- idem per udp

```
tcp flags { fin, syn, rst, psh, ack, urg, ecn, cwr }
```

- è possibile scrivere espressioni logiche anche con bit mask es:

```
tcp flags & (syn | ack) == syn | ack
```

```
icmp type <tipo>
```

nft – statement di tipo *verdict*

verdicts : alterano il flusso di controllo e decidono il destino dei pacchetti

- *accept*: accetta il pacchetto e interrompe la valutazione
- *drop*: elimina il pacchetto e interrompe la valutazione
- *queue*: accoda il pacchetto nello spazio utente e interrompe la valutazione
- *continue*: continua la valutazione del set di regole con la regola successiva.
- *return*: torna dalla catena corrente e continua alla regola successiva dell'ultima catena. In una catena di base è equivalente ad *accept*
- *jump <catena>*: continua alla prima regola di <catena>. Continuerà alla regola successiva dopo che è stata emessa un'istruzione *return*
- *goto <catena>*: simile a *jump*, ma dopo la nuova catena la valutazione continuerà all'ultima catena anziché a quella contenente l'istruzione *goto*

nft – statement per il monitoraggio

log

- può essere seguito da

- `level { emerg, alert, crit, err, warn, notice, info, debug }`
- `prefix <string>`

counter

- può inizializzare il contatore

- `packets <num_p> bytes <num_b>`

Nota: nft permette di eseguire più statement in una singola regola

- ovviamente uno solo può essere un verdict

- esempio loggare pacchetto vietato e scartarlo:

- `nft add rule ip foo bar tcp dport !=80 log prefix "forbidden: " drop`

nft – statement per NAT

`dnat to <dest_addr>`

`snat to <src_addr>`

`masquerade`

– opzionalmente seguito dalla nuova porta

- `to <port>`
- `to <min_port>:<max_port>`

nft - rappresentazione

Esaminando quanto generato dagli esempi precedenti con `nft list table foo` (o `nft list ruleset` per il dump completo) si visualizzerebbe

```
table ip foo {  
    chain bar {  
        type filter hook input priority filter; policy drop;  
        ct state established,related accept  
        ct state new tcp dport 22 accept  
    }  
}
```

Direttive in questa rappresentazione possono essere anche scritte in un file e attivate semplicemente con

```
nft -f <file>
```

È comune pulire il set di regole prima di configurare il packet filter, inserendo all'inizio del *file* il comando

```
flush ruleset
```

Netfilter connection tracking

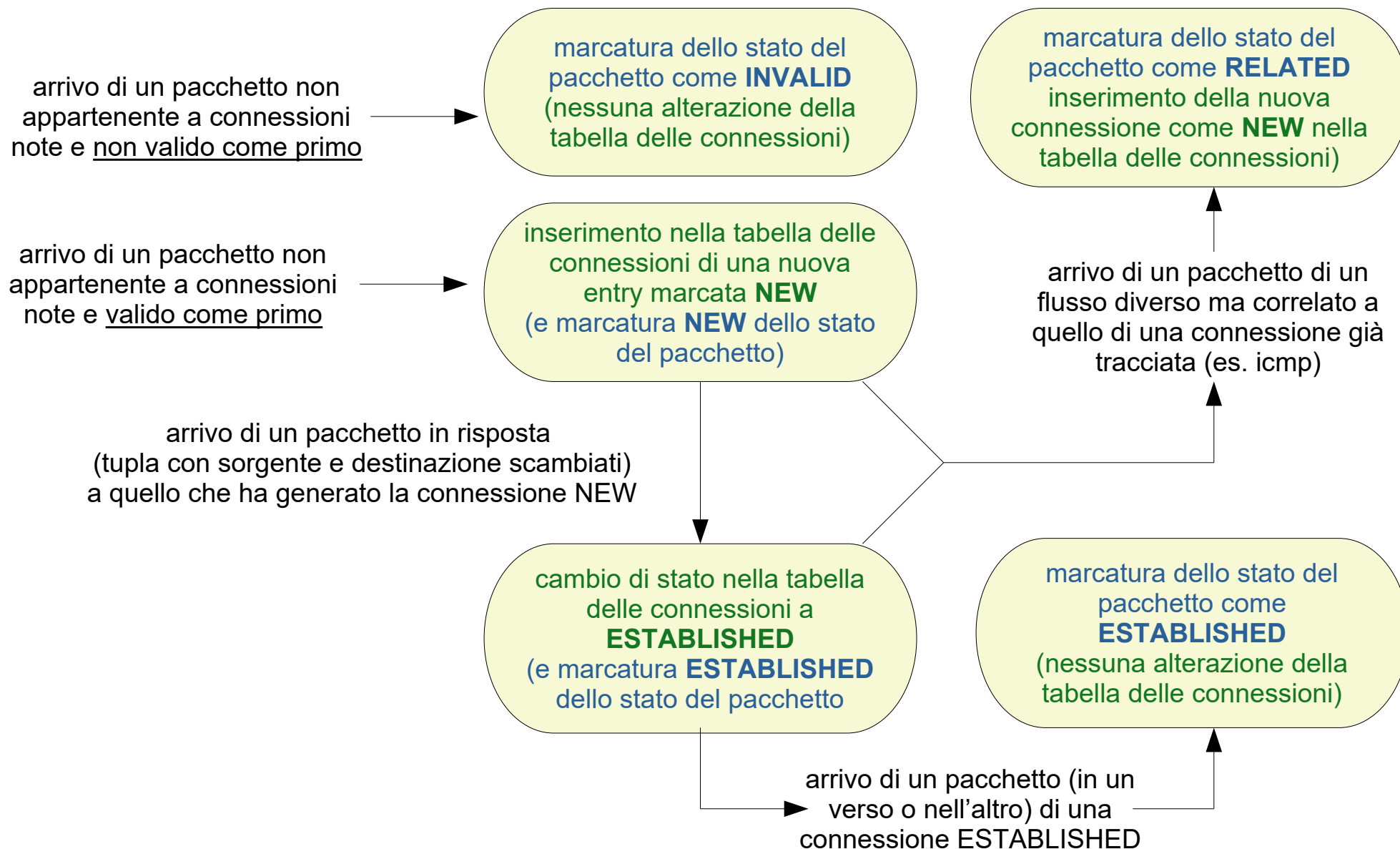
un'operazione fondamentale per l'efficacia del filtraggio, svolta da netfilter, è riconoscere lo stato dei pacchetti rispetto a una connessione

- operazione trasparente svolta da **conntrack**
- conntrack è una funzione invocata al raggiungimento dell'hook prerouting con priorità -200

si può imporre che un pacchetto salti le procedure di tracking

- l'azione deve essere svolta ovviamente prima dell'entrata del pacchetto in conntrack
- **IPTABLES** : usando target `-j CT --notrack` nella catena **PREROUTING** della tabella **raw**
- **NFTABLES** : usando lo statement `notrack` - non esistendo catene e tabelle predefinite, va specificato esplicitamente `hook prerouting priority -300`
 - più in generale: come assegnare meta-informazioni sui pacchetti per il connection tracking
https://wiki.nftables.org/wiki-nftables/index.php/Setting_packet_connection_tracking_meta_information

stati di tracciamento



Netfilter connection tracking

il connection tracker

- opera con una propria logica, indipendente dal fatto che il protocollo del pacchetto sia connection-oriented o connection-less
 - definiamo connessione semplicemente la tupla
<protocollo, ip sorgente, ip destinazione [, porta sorgente, porta destinazione]>
 - il comando `conntrack -L` mostra le entry della tabella delle connessioni
- riconosce pacchetti che iniziano nuove connessioni
- riconosce l'appartenenza di pacchetti a connessioni esistenti
- applica automaticamente alcune operazioni a tutti i pacchetti di una connessione (vedi NAT)
- permette di utilizzare lo stato della connessione come match

stati di tracciamento per nat

quando un pacchetto viene sottoposto ad address translation, alla sua connessione viene assegnato uno stato “virtuale” (aggiuntivo) SNAT o DNAT

due effetti automatici:

- i pacchetti della connessione nella stessa direzione vengono automaticamente modificati nello stesso modo
 - le regole della tabella **nat** operano quindi solo sul primo pacchetto della connessione, poi ci pensa **conntrack**
- i pacchetti della connessione in direzione opposta (le risposte) vengono automaticamente ripristinati con l'indirizzo originale
 - se sono risposte a pacchetti sottoposti a SNAT, l'indirizzo di destinazione viene ripristinato all'indirizzo sorgente originale
 - se sono risposte a pacchetti sottoposti a DNAT, l'indirizzo della sorgente viene ripristinato all'indirizzo destinazione originale
 - queste operazioni avvengono in conntrack, prima di qualsiasi altra analisi da parte di iptables → **tenerne conto nel filtraggio**

stateful filtering

dal punto di vista del filtraggio, il connection tracking è molto utile perché permette di utilizzare gli stati dei pacchetti per raffinare i match

- per accettare i pacchetti validi come iniziatori di una connessione (nella direzione lecita da un client verso un server) e seguenti
 - ***IPTABLES*** : `-m state --state NEW,ESTABLISHED`
- per accettare solo pacchetti validi come risposte a una connessione già iniziata (ad esempio da un server verso un client)
 - ***IPTABLES*** : `-m state --state ESTABLISHED`

NFTABLES

- esempio analogo a quest'ultimo di iptables:

```
table inet stateful_fw_demo {  
    chain IN {  
        type filter hook input priority filter; policy drop;  
        ct state established accept  
    }  
}
```

in generale: come utilizzare le meta-informazioni di connection tracking per gestire i pacchetti
https://wiki.nftables.org/wiki-nftables/index.php/Matching_connection_tracking_stateful_meta_information

nft advanced – set

È possibile definire insiemi di elementi da referenziare simbolicamente nelle regole: i **set**

creazione
del set

nome
del set

tipo degli elementi

- `ipv4_addr`: IPv4 address
- `ipv6_addr`: IPv6 address
- `ether_addr`: Eth address
- `inet_proto`: Inet protocol type
- `inet_service`: Internet service (read tcp port for example)
- `mark`: Mark type
- `ifname`: Net interface (eth0, eth1..)

```
nft add set ip foo goodlist { type ipv4_addr \;
```

```
nft add rule ip foo bar ip daddr @goodlist counter accept
```

```
nft add element ip foo goodlist { \  
    192.168.0.1, \  
    192.168.0.10 \  
}
```

referimento
al set

aggiunta di
elementi

nft advanced – map

Simili ai set, le **map** permettono di definire coppie nome-valore di cui fare lookup nelle regole

```
nft add table ip nat
nft add chain ip nat post { \
type nat hook postrouting priority 0\; }
nft add rule ip nat post snat ip saddr map { \
  1.1.1.0/24 : 192.168.3.11 , \
  2.2.2.0/24 : 192.168.3.12 \
}
```

ai pacchetti con questo IP sorgente
sarà dato questo nuovo IP sorgente

map “anonima” creata
direttamente nella regola

map con nome creata
analogamente ai set
(notare i due tipi chiave:valore

```
nft add map nat porttoip { type inet_service: ipv4_addr\; }
nft add element nat porttoip { 80 : 1.1.1.1, 22 : 2.2.2.2 }
```

nft advanced – verdict maps

Le vmaps sono un caso particolare di mappe con cui specificare in forma compatta verdetti diversi per valori alternativi di un match

Esempi:

- salto a catene custom (ipotizzando di avere già creato le catene *tcp-chain* e *udp-chain*)

```
nft add rule ip filter input ip protocol vmap \  
{ tcp:jump tcp-chain, udp : jump udp-chain }
```

- con porte o indirizzi

```
nft add rule ip foo bar ip saddr vmap \  
{ 1.1.1.1-1.1.1.5 : accept, 2.2.2.2/24 : drop }
```

```
nft add rule ip foo bar tcp dport vmap \  
{ 8000-8099 : accept, 22-25 : drop }
```

valore di match
corrispondente al
tipo di parametro
dichiarato

verdict